arXiv:2605.16024v2 [cs.AI] 17 Aug 2026

SCREENSEARCH: Uncertainty-Aware OS Exploration

Michael Solodko[*†] **Justin Wagle**[*†]
Microsoft[†]
[*]Equal contribution.

Abstract

Desktop GUI agents operate under partial observability: nearly identical screens can hide different workflow states, and the same interaction can therefore have different outcomes. We present SCREENSEARCH, a distributed exploration system that builds a shared deduplicated screen graph and treats matched-action outcome dispersion as a structural ambiguity signal. A one-step PUCT graph-bandit combines state and edge novelty with movement toward lower-dispersion states; it executes one action per selection and performs no simulated rollouts or multi-step backup. Across 11 applications, SCREENSEARCH records 1,007,406 screenshots, 39,902 unique observations, and 31,146 merged graph states. In a matched evaluation with 50 replay starts in each of Notepad, File Explorer, and LibreOffice Calc, PUCT raises macro-average new-state AUC from 4.820 to 50.487 over greedy selection using the same prior. Adding the ambiguity term to novelty raises state AUC by 8.9%, while trading off against edge coverage, so we do not claim uniform dominance. Controlled hidden-state, timing-noise, and reversible-probe tests clarify what the ambiguity proxy detects and when lower dispersion constitutes useful disclosure. Finally, an OCR audit shows that the representation need not be tied exclusively to accessibility metadata, and a downstream policy study shows that counterfactual screen supervision improves state sensitivity and offline recovery. These results support uncertainty-aware state exploration while making its causal and deployment limits explicit.

1 Introduction

Computer-use environments are large, heterogeneous, and partially observable. A single desktop observation rarely determines the underlying workflow state: near-identical screens can reflect different permissions, clipboard contents, focus, authentication context, pending asynchronous events, or hidden application modes. The challenge is therefore not only to choose a plausible next UI action, but also to decide when the available observation is sufficient for commitment and when another interaction should gather evidence. A locally reasonable action can otherwise push an agent into a brittle part of the workflow before the relevant latent cases have been distinguished.

Most computer-use agents are evaluated through end-to-end instruction following and operate stepwise or with a short receding horizon. That paradigm is effective for task completion, but offers limited support for explicitly mapping an unfamiliar application or uncovering the hidden structure behind repeated observations. We instead study open-ended computer/OS exploration, where the product of interaction is a reusable graph of screens, actions, and observed outcomes. In this setting, observations are better treated as hypotheses about workflow state than as complete state descriptions.

Two exploration signals are complementary. *Novelty* rewards discovering new deduplicated states and labeled transitions, thereby expanding the known frontier. *Ambiguity* measures whether matched actions from similar observations lead to inconsistent successors. Novelty alone can over-reward superficial visual change; ambiguity alone can remain within a small uncertain region or prefer an

Preprint.

uninformative escape. Their useful balance depends on the proposal mechanism, the quality of state identity, and the cost of committing while outcomes remain unreliable.

We introduce SCREENSEARCH, a distributed system that turns UI observations into location-aware structural and semantic features, retrieves candidate matches with a hybrid sparse–dense index, verifies near-duplicates, and shares a state graph across isolated VM workers. This global memory supports revisit detection, loop avoidance, action-level statistics, and empirical ambiguity estimation. Over the graph, a one-step PUCT selector balances plausible actions, frontier expansion, and movement toward lower-dispersion states. We call it a *PUCT graph-bandit*: it borrows the PUCT selection rule, but executes one environment action per selection and performs neither internal simulations nor multi-step return backup.

The same interaction traces support learned action proposals and downstream screen-conditioned policies. We retain structured screen–action windows, remove loop- and no-op-dominated traces, and construct same-task/different-screen counterfactual pairs. This connects exploration to policy learning without requiring the explorer itself to become a trajectory-level planner: learned proposals guide which executable actions deserve attention, while downstream models test whether the collected data teaches actions that depend on the current screen.

We evaluate the full system at two complementary scales. A 5,524-VM-hour corpus across 11 applications measures state discovery and operational behavior. A matched 150-start suite across Notepad, File Explorer, and LibreOffice Calc isolates selector, objective, proposal, and aggregation effects. Controlled clipboard, timing, and reversible-action studies distinguish hidden-condition dispersion from stochastic execution and test whether a candidate probe improves a later commitment. Finally, OCR and downstream-policy studies examine whether the representation and collected traces transfer beyond the exact exploration backend.

Our contributions are:

- a scalable distributed screen graph built from location-aware UI features, hybrid sparse–dense retrieval, bounded near-duplicate verification, and deterministic action identities;
- a matched-action outcome-dispersion proxy and one-step PUCT objective for uncertainty-aware exploration, accompanied by causal controls and an operational probe criterion;
- large-scale exploration across 11 desktop applications plus matched cross-application selector, objective, prior, state-aggregation, and sensitivity analyses; and
- a trace-processing bridge from exploration to learned proposals and downstream policies, with evidence that OCR can supply an alternative representation and counterfactual screen supervision improves offline grounding and recovery.

2 Related work

Interactive web and OS agents. Computer-use benchmarks range from programmatic web interfaces to realistic websites and full desktop environments (Liu et al., 2018; Deng et al., 2023; Zhou et al., 2023b; Xie et al., 2024). They expose brittleness under long horizons, UI variation, and real execution constraints. We share the goal of robust interaction, but focus on the complementary systems problem of discovering reusable state structure when observations are aliased, rather than optimizing a single task-completion episode.

Perceptual aliasing and active sensing. Perceptual aliasing describes observations that fail to distinguish states requiring different behavior (Chrisman, 1992). Active sensing and active perception choose actions partly for the evidence they reveal (Veiga and Renoux, 2023; Bajcsy et al., 2018). These methods often optimize entropy or mutual-information objectives under explicit belief-state or world-model assumptions. SCREENSEARCH adopts their information-seeking motivation without claiming a Bayesian posterior: its ambiguity score is a scalable empirical reliability proxy over matched action outcomes.

Planning and exploration. POMCP and AlphaZero-style search show how visit statistics and priors can guide decisions in large spaces (Silver and Veness, 2010; Silver et al., 2017, 2018). Intrinsic-motivation methods reward prediction error, novelty, visitation, or learned uncertainty (Pathak et al.,

2017; Burda et al., 2018; Sekar et al., 2020). GUI environments add a practical difficulty: visually similar observations can hide different workflow states, so state novelty need not imply reliable action outcomes. Our selector uses the PUCT exploration bonus over a transposition graph, but updates it only with the observed immediate transition reward. It is therefore neither multi-step MCTS nor belief-state planning.

Stepwise agents and retrieval-augmented memory. Language agents commonly interleave short-horizon reasoning and tool actions (Yao et al., 2022); reflection and deliberate-search variants improve robustness by reconsidering candidate continuations (Ahn et al., 2022; Shinn et al., 2023; Zhou et al., 2023a). Other systems retrieve relevant prior experience as an external memory (Kagaya et al., 2024). SCREENSEARCH uses retrieval for a different but related purpose: it identifies revisits and near-duplicates, pools matched-action evidence, and exposes shared graph statistics to many workers. The resulting memory is part of the exploration state rather than only prompt context.

Trajectory learning. Offline sequence models treat control as conditional generation, while diffusion policies and planners represent multimodal actions through denoising (Chen et al., 2021; Janner et al., 2022; Chi et al., 2023). We use SCREENSEARCH traces both to learn proposal priors and to study screen-conditioned action prediction. In contrast to using diffusion as the explorer's search procedure, the reported exploration policy remains the one-step graph-bandit; the downstream diffusion evaluation is an offline, teacher-forced proxy rather than live task completion.

3 Computer/OS state exploration

Let $G_t = (V_t, E_t)$ be the global deduplicated state graph before interaction t. A realized labeled edge is (s, σ, s'), where s and s' are merged states and σ is a deterministic action signature. A transition is state-novel when $s' \notin V_t$ and edge-novel when $(s, \sigma, s') \notin E_t$.

Screen novelty and the exploration frontier. State and edge novelty capture different forms of progress. A new state expands the set of known observations, whereas a new labeled edge reveals another reachable outcome or another way to reach an existing state. Retaining both matters in GUIs: two actions may converge to the same dialog while exposing different interaction affordances, and a visually new screen may be reached through a transition already represented elsewhere in the graph. Novelty is always evaluated against the shared graph snapshot immediately preceding an action, so concurrent workers contribute to a common, continually changing frontier.

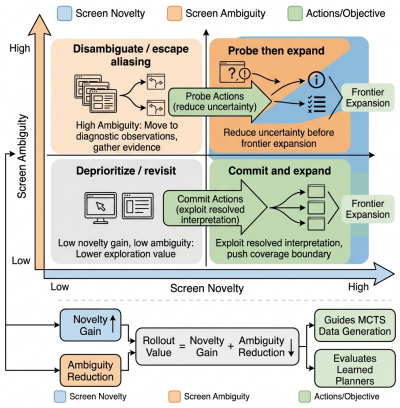


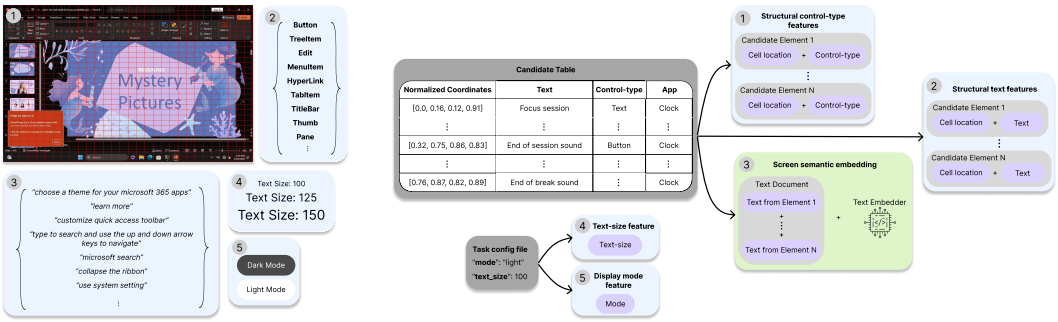
Figure 1: Complementary exploration signals: novelty expands coverage, while ambiguity identifies empirically unreliable matched-action outcomes.

Action identity. The exact signature is

`template_id@version::target_signature::arg_signature`.

The target signature is `cell|role|label|application`: the target center is quantized on a 30×30 active-window grid, and role, label, and application strings are normalized. Arguments use sorted compact JSON. Clicks therefore retain target location and semantics; type actions additionally retain the exact text. Candidate IDs and raw pixels are excluded because they are unstable. This prevents graph counts and ambiguity pools from collapsing parameter-distinct interactions. Examples appear in Section A.

Screen ambiguity. Let $\mathcal{O}(s)$ be the near-duplicate observation cluster assigned to state s, and let $\Sigma(s)$ be its observed signatures. For each signature, $P(s' \mid s, \sigma)$ is the empirical successor



(a) Discrete feature universes.

(b) Feature extraction.

Figure 2: The active representation combines spatial structural atoms, normalized text, a dense semantic vector, and screen-level metadata.

distribution and $n_{s,\sigma}$ its sample count. We define

$$D(s) = \sum_{\sigma \in \Sigma(s)} w_{s,\sigma}\, \bar{H}(P(s' \mid s, \sigma)), \qquad w_{s,\sigma} = \frac{n_{s,\sigma}}{\sum_{\tilde{\sigma}} n_{s,\tilde{\sigma}}}, \tag{1}$$

$$\rho(s) = \frac{n_s}{n_s + \kappa}, \qquad u(s) = \rho(s)D(s) + (1 - \rho(s))u_0, \tag{2}$$

where $\bar{H} \in [0, 1]$ is normalized entropy and $n_s = \sum_\sigma n_{s,\sigma}$. Reported runs use $\kappa = 20$ and $u_0 = 0.5$. The score detects empirical outcome dispersion; it does not identify its cause from a single observation. Hidden workflow state, asynchronous timing, execution noise, and merge errors can all raise $u(s)$.

Joint exploration objective. Pure novelty can repeatedly chase cosmetic changes, while pure ambiguity reduction may remain local or simply leave an uncertain region. We therefore reward graph expansion and positive ambiguity reduction together. This is a structural information-seeking objective rather than an explicit expected-information-gain calculation: it is designed to remain computable from the transition log shared by the workers, without learning a latent dynamics model or maintaining a belief distribution over every application state.

Probe versus escape. We do not label an action a probe merely because it reaches a low-dispersion destination. Operationally, a probe must reveal information that improves a later latent-dependent commitment relative to a matched non-disclosing action. This criterion is evaluated retrospectively in the controlled study in Section 9.3; it is not a hard-coded action class used by the selector.

4 Representation, retrieval, and state identity

4.1 Screen representation

Each screen observation S is represented from its UI Automation (UIA) element set $\mathcal{E}(S)$. An element supplies a bounding box, control role, and optional text. We normalize the role and text, quantize the element center into a shared spatial grid, and create cell–control and cell–text atoms. These atoms retain local layout and lexical cues while tolerating small coordinate shifts. Display mode and text-size bin are stored as screen-scoped metadata, and concatenated normalized screen text is embedded with OpenAI `text-embedding-3-small`.

The resulting mixed signature has three roles. Sparse atoms support interpretable lexical retrieval and bounded overlap tests; the dense vector recovers semantically related screens whose surface tokens differ; and the metadata prevents implausible matches across configured display contexts. UIA is the backend used for the reported online runs, but indexing, graph construction, and ambiguity estimation consume the resulting feature sets rather than invoking UIA-specific logic. This separation makes it possible to substitute another observation encoder, as tested by the OCR audit in Section E.

4.2 Hybrid retrieval and bounded verification

Given a query S_q and an accumulated corpus $\mathcal{D}$, exact display-mode and text-size filters first define an eligible pool. Within that pool, BM25 ranks serialized sparse atoms and HNSW approximate nearest-neighbor search ranks dense vectors. Reciprocal Rank Fusion combines the two lists, and the fused top 200 proceed to verification. This two-stage design separates high-recall candidate generation from the stricter decision that alters graph identity.

For verification, let $T_{\mathrm{ct}}(S)$ and $T_{\mathrm{txt}}(S)$ be the sets of cell–control and cell–text atoms. We compute their Jaccard overlaps separately and use the bounded score

$$J_{\mathrm{sparse}}(S_q, S) = 0.5J(T_{\mathrm{ct}}(S_q), T_{\mathrm{ct}}(S)) + 0.5J(T_{\mathrm{txt}}(S_q), T_{\mathrm{txt}}(S)), \tag{3}$$

with merge threshold $\tau = 0.93$. A query joins the highest-ranked candidate that passes this test; otherwise it becomes a new graph state. Dense similarity can therefore improve recall but cannot by itself merge two observations. This choice makes the definition of state identity explicit and keeps its downstream effects on novelty, visits, and ambiguity auditable. The complete pipeline is diagrammed in Fig. 5; sensitivity to τ and exact-matching alternatives is reported in Section D.

5 One-step PUCT action selection

At state s, the system enumerates executable action signatures and selects

$$\sigma^* = \arg\max_{\sigma} \left[Q(s,\sigma) + c_p P(\sigma \mid s) \frac{\sqrt{\sum_{\tilde{\sigma}} N(s,\tilde{\sigma})}}{1 + N(s,\sigma)} \right], \tag{4}$$

with $c_p = 1.0$. The selected action is executed once in the VM. Relative to the graph snapshot immediately before execution, its observed reward is

$$r(s,\sigma,s') = \lambda_{\mathrm{state}} \mathbb{I}[s' \notin V_t] + \lambda_{\mathrm{edge}} \mathbb{I}[(s,\sigma,s') \notin E_t] + \lambda_{\mathrm{amb}} [u(s) - u(s')]_+, \tag{5}$$

where all three coefficients equal 1.0. The shared store increments $N(s,\sigma)$ and updates $Q(s,\sigma)$ as the running mean of this immediate reward. There are no simulated rollouts, no tree backpropagation, and no discounted return; the configured $\gamma = 1.0$ is therefore unused.

This update has a useful distributed interpretation. Workers that reach the same merged state contribute to the same action counts and empirical values even when they arrived through different trajectories. Frequently tried low-reward signatures are gradually suppressed by their visit denominator, whereas underexplored actions retain a larger exploration bonus. Because the destination is observed in the real VM, the statistics incorporate actual application behavior rather than a learned transition model. Their reliability nevertheless remains coupled to state merging and action canonicalization, which motivates the exact definitions in Sections 3 and 4.

Unless otherwise stated, the deterministic proposal score is

$$\tilde{P}(\sigma \mid s) = 1 + 0.2\,|\mathrm{tokens}(g) \cap \mathrm{tokens}(\mathrm{label}(\sigma))| + 0.15\mathbb{I}[\mathrm{editable}(\sigma) \wedge \mathrm{type}(\sigma)], \tag{6}$$

normalized over current candidates, where g is the exploration instruction. We also evaluate uniform and learned diffusion priors. The proposal and the accumulated PUCT statistics make separable contributions in Section 9.

6 Proposal priors from exploration traces

The PUCT graph-bandit runs produce a large corpus of short interaction traces that expose aliasing, loops, and failure modes. The main contribution is the exploration pipeline itself, but the traces also support lightweight proposal priors over executable action signatures conditioned on the current screen. A proposal changes only $P(\sigma \mid s)$ in Eq. (4); it is not treated as a standalone planner with a separate control objective, and the graph-bandit remains responsible for frontier-sensitive selection and ambiguity reduction.

The original corpus-building profile uses `uniform_prior@1`. An optional deterministic `heuristic@1` profile ranks plausible signatures using the local structural and lexical cues in Eq. (6);

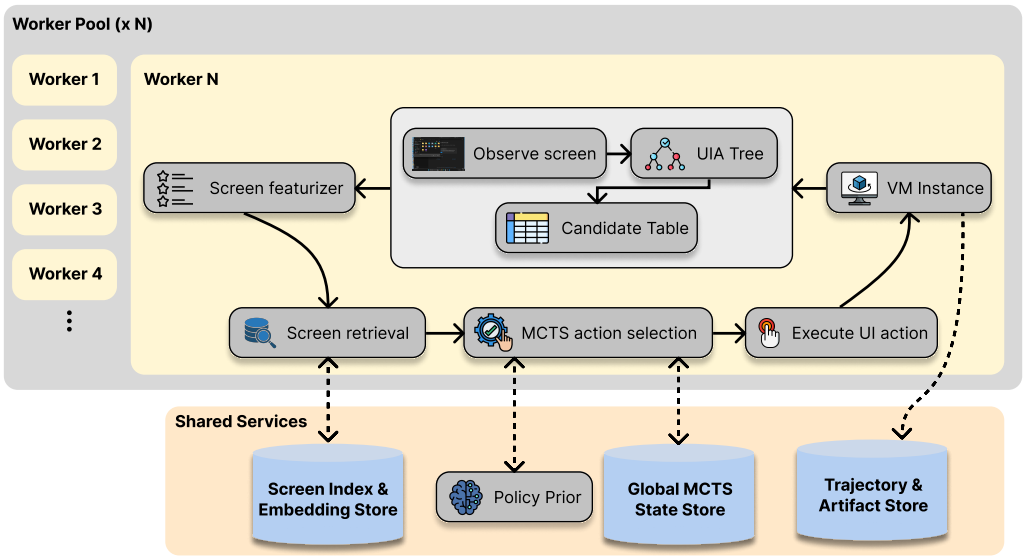


Figure 3: Original distributed OS-exploration system overview. In the terminology used throughout this paper, the diagram's legacy "MCTS action selection" and "Global MCTS State Store" labels denote the one-step PUCT selector and its shared graph-statistics store, respectively; the reported system performs neither simulated rollouts nor tree backpropagation.

the matched three-application selector study uses this profile for both PUCT and greedy selection so that the selector is isolated. We additionally train a conditional diffusion proposal from retained traces. Comparing uniform, deterministic, and learned proposals lets us separate local proposal quality from the shared graph statistics accumulated by search.

Each worker records the screen representation, candidate table, canonical action, successor identity, and exploration statistics. Post-processing removes repeated cycles and near-no-op segments, then tokenizes retained actions by operation, target role and text, quantized position, and optional argument. The complete schema and marginal frequencies appear in Sections G.3 and G.5. A separate downstream experiment later uses the same traces for screen-conditioned action prediction; future screens are used only by training auxiliaries, and its multi-step metrics remain teacher-forced offline diagnostics.

7 OS exploration setup

We evaluate ScreenSearch as a large-scale desktop GUI exploration and data-generation system. For each application, we report VM-hours, cross-application captures, total screenshots, post-processed unique observations, merged graph states, and extracted three-step traces. The architecture in Fig. 3 separates replicated VM workers from the shared index, PUCT statistics, and artifact stores.

Large-scale corpus. Exploration runs begin from clean Windows VM snapshots, followed by a full reset between runs so that latent application state does not accumulate across trials. Seeded content, light or dark display mode, application size, and window placement vary across configurations to expose different observable regions. The large-scale collection totals 5,524 recorded VM-hours across 11 applications (Table 2).

Action space and cross-application capture. Mouse clicks and keyboard typing are instantiated from the current UIA candidate table. Clicks target candidate elements; type actions are available only for text-editable targets and retain their exact argument in the action signature. Candidates are restricted to elements attributed to the target application. When an interaction opens a system dialog, file picker, authentication surface, or auxiliary application, the transition is logged as a cross-application capture rather than silently folded into the target-app state graph.

Table 1: State-matching hyperparameters used for OS exploration.

Hyperparameter	Value
Similarity threshold τ	0.93
Cell×control-role Jaccard weight	0.5
Cell×text Jaccard weight	0.5

Trajectory extraction. Each run yields a raw interaction trace. Post-processing removes segments dominated by short loops and near-no-op actions, then extracts short windows that exhibit sustained observable progression. Structured UI metadata is retained alongside screenshots so that task descriptions, grounded element labels, and counterfactual training pairs can be regenerated offline without rerunning the VM exploration. Prompt templates and action-token construction appear in Sections G.2 and G.3.

Deduplication hyperparameters. The similarity threshold determines whether a new observation joins an existing state or is indexed as new. The two component weights balance structural control-role evidence against spatial text evidence in the bounded duplicate score. We use the same values throughout the reported corpus and controlled selector runs.

Matched policy suite. The primary controlled study uses 50 verified replay starts in each of Notepad, File Explorer, and LibreOffice Calc, stratified as 17 low-, 16 medium-, and 17 high-ambiguity starts. A deterministic prefix replays each state from a clean snapshot. Conditions share starts, candidates, seeds, 50-action budgets, and all non-ablated settings. "New states" and "new edges" count coverage beyond the initialized replay artifact. We report means over the 50 starts per application. The original three-start Notepad diagnostic and its standard deviations are retained in Section B, but are not used as cross-application evidence.

Metrics. Final counts and area under the cumulative discovery curve (AUC) measure additional state and edge coverage. Revisit rate is the fraction of steps returning to an episode-visited state. Loop/stall rate is the fraction belonging to a repeated short cycle or a no-progress run. High-ambiguity occupancy is the fraction of visited states above the fixed ambiguity threshold used for the replay suite. Net ambiguity change is the final minus initial $u(s)$.

Reactive baseline interface. Reactive models receive the instruction, active-window metadata, exploration counters, and at most 60 UIA candidates with ID, label, role, rectangle, clickability, and editability. They emit one validated JSON action from {`click`, `type`, `wait`}. Full prompting, sampling, and context details are in Section A.

8 Large-scale OS exploration results

8.1 Overall performance

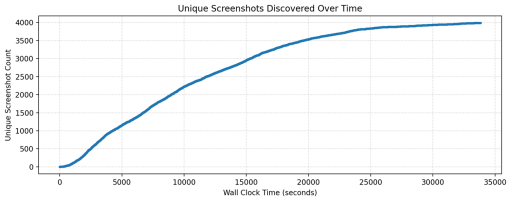
The 39,902 unique observations in Table 2 are post-processed screenshot identities before the final bounded state merge; applying the active representation and threshold produces 31,146 graph states. State discovery rates range from 1.03% in Notepad to 7.83% in LibreOffice Calc, reflecting differences in interface variability and reachable seeded workflows. The 43 cross-application captures show that target-restricted actions can still surface operating-system dialogs and auxiliary applications, while the 55,770 extracted traces quantify the short interaction windows retained for subsequent analysis.

State discovery over time. The cumulative curve in Fig. 4a rises rapidly while workers traverse high-branching UI regions, then gradually flattens as exploration encounters more revisits and near-duplicates. The curve is therefore useful alongside the final corpus totals: it distinguishes sustained discovery from a system that produces most of its coverage in a brief initial phase.

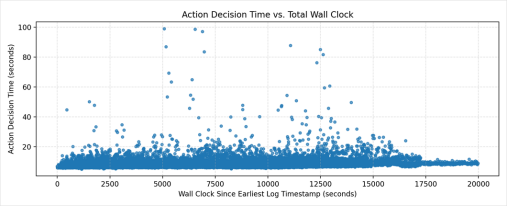
Efficiency, state coherence, and trace extraction. Per-step action-decision latency remains stable over the collection run in Fig. 4b, with isolated spikes consistent with transient VM or application delays rather than steady growth with index size. The changed-pixel distribution in Fig. 4c is

Table 2: Large-scale OS exploration corpus. "Unique obs." are post-processed screenshot identities; "graph states" apply the final bounded merge. The last column counts extracted length-three traces.

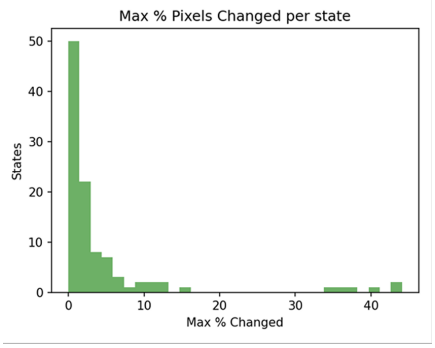
Application	VM hours	Cross-app	Screenshots	Unique obs.	Graph states	State rate (%)	3-step traces
Clock	214	1	42,500	1,366	1,366	3.21	3,701
File Explorer	366	8	52,763	3,985	3,985	7.55	2,662
Microsoft Excel	663	6	86,025	2,585	2,585	3.00	4,324
Microsoft Word	1,377	1	213,613	8,759	8,759	4.10	10,777
Microsoft PowerPoint	1,809	1	466,115	8,124	8,124	1.74	15,834
Notepad	110	1	30,445	314	314	1.03	612
Settings	460	20	53,037	2,310	2,310	4.36	13,457
VS Code	250	1	39,533	1,944	1,944	4.92	1,995
LibreOffice Writer	92	2	7,840	2,420	587	7.49	924
LibreOffice Calc	85	1	7,841	2,642	614	7.83	396
LibreOffice Math	98	1	7,694	5,453	558	7.25	1,088
Overall	**5,524**	**43**	**1,007,406**	**39,902**	**31,146**	**3.09**	**55,770**



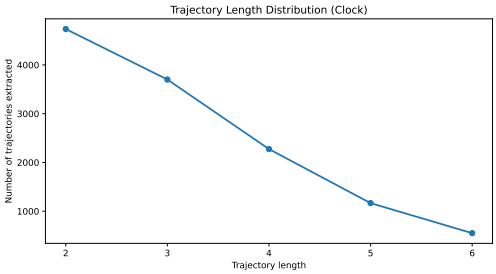
(a) Cumulative unique observations over wall-clock time.



(b) Per-step decision latency as the corpus grows.



(c) Maximum changed-pixel fraction within merged states.



(d) Extracted Clock trajectories by length.

Figure 4: Large-scale OS-exploration diagnostics. The plots restore the temporal discovery, scaling, state-coherence, and trace-extraction views used to interpret the corpus.

concentrated near zero, providing an independent visual-coherence diagnostic for merged states. Finally, Fig. 4d shows the expected short-trace skew after filtering loop-heavy and near-no-op segments.

Prior-guided corpus exploration. In the original corpus-building comparison, replacing uniform action weights with the deterministic heuristic prior raises unique-state discovery from 1.74% to 4.10%. This comparison is descriptive because it comes from the large-scale generation profile rather than the matched replay suite; the controlled learned-prior and fixed-prior selector comparisons are reported separately in Section B and Table 4.

9 Exploration under uncertainty: policy evaluation

Large-scale corpus statistics establish coverage but do not isolate the action selector. We therefore evaluate SCREENSEARCH as a policy problem under uncertainty: useful behavior should expand the

Table 3: Proposal prior effect in the original corpus-building comparison.

Proposal prior	Unique-state discovery rate (%)
Deterministic heuristic	4.10
Uniform	1.74

Table 4: Matched cross-application selector comparison. Both rows per application use the same deterministic heuristic prior. Values are means over 50 starts.

Application	Selector	New states @ 50	State AUC	Edge AUC	Net Δu	Revisit
Notepad	PUCT	1.000	32.900	361.960	0.0608	0.9796
	Greedy	0.240	11.560	78.500	-0.0505	0.9951
File Explorer	PUCT	3.140	98.080	1,076.400	0.0351	0.9357
	Greedy	0.040	1.920	55.640	0.0691	0.9992
LibreOffice Calc	PUCT	0.580	20.480	709.720	-0.0711	0.9882
	Greedy	0.020	0.980	49.940	0.0182	0.9996
Macro-average	PUCT	**1.573**	**50.487**	**716.027**	0.0083	**0.9678**
	Greedy	0.100	4.820	61.360	0.0123	0.9980

discovered frontier while avoiding repeated commitment to already known or low-yield branches. The primary replay-start suite measures additional state and edge coverage, ambiguity trajectories, revisits, and stalls under matched executable candidates and budgets.

9.1 Matched cross-application selector comparison

Table 4 holds proposal probabilities fixed and changes only greedy selection versus the PUCT statistics in Eq. (4). PUCT improves new-state and new-edge AUC in every application. At the macro level, new states rise from 0.100 to 1.573 and state AUC from 4.820 to 50.487, while revisit falls from 0.9980 to 0.9678. The high absolute revisit rates reflect difficult starts initialized from an existing replay artifact; only additional coverage receives credit.

Proposal quality remains important. On the original matched Notepad diagnostic, replacing a uniform prior with a learned diffusion prior raises PUCT frontier states from 10.00 to 14.33 and state AUC from 253.33 to 406.67, while reducing revisit from 0.796 to 0.708 and loop/stall from 0.440 to 0.253 (Table 11). Thus learned proposals and shared PUCT statistics are complementary rather than interchangeable.

9.2 Objective and component ablations

Table 5 uses the same three-application starts and heuristic prior except where named. Relative to novelty-only, the full objective raises state AUC by 8.9% and modestly lowers revisit and loop/stall, but novelty-only has higher edge AUC and lower high-ambiguity occupancy. Prior-only and uniform-prior variants find more raw states but also spend more time in high-ambiguity states. These results support a coverage–ambiguity trade-off, not uniform dominance of the full reward. Exact matching changes the definition of state identity, so its larger raw count is not interpreted as better semantic coverage.

One-factor sensitivity results are reported in Section D. Increasing τ reduces false merges but fragments states, κ has a comparatively modest effect around 20, and u_0 is a meaningful operating choice. Pair F1 uses persisted UIA IDs as a matcher diagnostic, not independent semantic ground truth.

9.3 Controlled ambiguity and probe validation

The same visible Notepad state was reset with clipboard value `alpha` or `beta`; Paste is deterministic within either variant but differs when variants are pooled. An engineered delayed clipboard update instead varies within one fixed condition. Table 6 shows the intended decomposition: known hidden

Table 5: Three-application macro-average component ablations under matched starts, candidates, seeds, and budgets.

Configuration	New states	State AUC	Edge AUC	High-u occ.	Revisit	Loop/stall
Full: novelty + ambiguity	1.573	50.487	716.027	0.6568	0.9678	0.8593
Novelty only	1.427	46.347	901.533	0.4907	0.9705	0.8828
Ambiguity only	1.047	34.300	761.987	0.6818	0.9786	0.9000
Prior only	2.233	71.867	982.900	0.7068	0.9544	0.8153
Full + uniform prior	4.733	132.527	1,094.720	0.7983	0.9028	0.6937
Full + exact matching	2.100	67.847	997.240	0.6038	0.9570	0.8651

Table 6: Controlled matched-action dispersion.

Source	Trials	Pooled	Within	Between
Hidden clipboard variants	20	1.000	0.000	1.000
Asynchronous race	10	0.971	0.971	0.000

state produces between-condition dispersion, while timing noise remains within condition. The scalar $u(s)$ alone does not perform this causal attribution; repeated condition labels are required.

We then test probe utility with 20 runs per condition. A reversible Paste discloses which hidden clipboard token is present; Undo restores the document before the policy types the corresponding commitment token. Opening and closing the File menu reaches an equally repeatable, low-dispersion destination but reveals nothing. Paste raises commitment accuracy from 0.500 to 1.000; menu navigation leaves it at 0.500 (Table 7). This is why lower destination dispersion alone is insufficient to call an action a probe.

An audit of 1,588 repeated state–action groups across the three applications provides a less synthetic check: nearly all measured dispersion lies between recorded contexts rather than within a fixed context (Table 12). This supports the proxy's usefulness while preserving the caveat that unrecorded timing and merge noise can still contribute.

9.4 Observation representation and downstream use

The OCR audit re-encodes the same 1,200 screenshots with UIA features or RapidOCR 1.4.4 using PP-OCRv4 (RapidAI Team, 2025; PaddleOCR Team, 2023). UIA obtains same-state pair F1 0.740 versus 0.646 for OCR, while OCR obtains recall@10 0.409 versus 0.381, false-merge rate 0.007 versus 0.010, and successor consistency 0.958 versus 0.892. Because reference labels are existing SCREENSEARCH IDs, the comparison favors UIA; nevertheless, OCR remains a viable input to the same retrieval and grouping pipeline (Section E).

For downstream evaluation, we slice Windows Settings traces into 22,028 three-step screen–action windows with 8,118 task descriptions; 93.99% contain a state change. We compare the same step-conditioned diffusion architecture trained with and without counterfactual pairs that hold task text fixed while changing the screen and target action. The split is task-template-disjoint and uses 1,000 frozen prompts. Future screens are used only for training auxiliaries, not action prediction.

Counterfactual supervision preserves one-step match while increasing screen sensitivity by 3.70 points, state-change sensitivity by 4.50 points, offline three-step success from 0.85% to 1.74%, and recovery from 21.68% to 24.35%. The policy changes its prediction on 66.52% of same-task/different-screen pairs. These results show that the traces can train screen-dependent behavior; they do not establish live end-to-end task completion.

10 Limitations and broader impacts

The ambiguity score is an empirical structural proxy, not a semantic posterior. It detects unreliable matched-action outcomes but cannot determine from one observation whether dispersion comes from hidden workflow state, asynchronous execution, stochasticity, or retrieval and merge errors. The controlled decomposition requires repeated trials or condition labels that will often be unavailable in

Table 7: Probe-versus-escape control, 20 runs per condition.

Condition	Disclosure	Destination dispersion	Commitment accuracy
No probe	0.000	–	0.500
Reversible Paste + Undo	1.000	0.000	1.000
File-menu navigation	0.000	0.000	0.500

Table 8: Downstream screen-conditioned policy results. Sensitivities are accuracy drops after screen shuffling. Three-step success and recovery are teacher-forced offline proxies, not live GUI success.

Training setup	Step match	Screen sens.	Change sens.	Offline 3-step	Recovery
With counterfactual pairs	52.60%	22.50 pts	23.45 pts	1.74%	24.35%
Without counterfactual pairs	53.10%	18.80 pts	18.95 pts	0.85%	21.68%

deployment. Likewise, a transition to lower $u(s)$ is not necessarily informative; the probe experiment evaluates disclosure through a later commitment, whereas the general selector optimizes only the proxy in Eq. (5).

The primary matched policy study covers three applications and a 50-action budget, despite the corpus spanning 11 applications. UIA metadata is effective for conventional desktop controls but can be missing for custom-rendered, canvas-based, or continuously changing interfaces. The OCR audit tests an alternative representation offline on stored screenshots; it does not establish equivalent online performance. The PUCT graph-bandit is one-step and therefore cannot by itself reason about delayed information value or irreversible multi-action consequences. Its quality remains coupled to candidate generation, replay fidelity, the retrieval index, and action-signature design.

The downstream metrics are teacher-forced offline proxies and the absolute three-step success rates are low; live task completion, safety, and recovery remain open. Large-scale GUI exploration can also expose sensitive text or execute destructive actions if deployed outside a sandbox. Our experiments use isolated clean-snapshot VMs, restrict candidates to the target application, and log cross-application transitions. Any external deployment or artifact release should additionally redact secrets, require explicit authorization, and gate irreversible actions. These safeguards limit direct user harm but do not eliminate dual-use risks from more capable autonomous computer control.

11 Conclusion

SCREENSEARCH treats desktop exploration as the joint problem of building a reusable state graph and recognizing when matched actions have unreliable outcomes. Its representation and hybrid retrieval system provide stable state identity at corpus scale, while the shared PUCT graph-bandit turns observed frontier and ambiguity signals into coordinated action statistics across workers. Matched experiments show that those statistics improve additional coverage over greedy use of the same prior, and the objective and proposal ablations reveal a genuine coverage–ambiguity trade-off rather than a universally dominant configuration. Controlled tests clarify both the value and limits of the dispersion proxy. Finally, the learned-prior, OCR, and downstream-policy studies show how the resulting graph and traces can support other observation encoders and screen-grounded models. Explicit state identity, proposal quality, and uncertainty-aware statistics are therefore complementary tools for deciding when a desktop agent should continue exploring before it commits.

References

M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, K. Jeffrey, S. Jesmonth, N. Joshi, R. Julian, D. Kalashnikov, S. Levine, L. Lu, C. Parada, K. Rao, U. Rückert, J. Schneider, K. Sheng, P. Sermanet, S. Sriram, A. Stone, T. Tan, H. Tran, V. Vanhoucke, F. Xia, T. Xu, and A. Zeng. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022. URL https://arxiv.org/abs/2204.01691.

R. Bajcsy, Y. Aloimonos, and J. K. Tsotsos. Revisiting active perception. *Autonomous Robots*, 42(2):177–196, 2018. doi: 10.1007/s10514-017-9615-3. URL https://doi.org/10.1007/s10514-017-9615-3.

Y. Burda, H. Edwards, A. J. Storkey, and O. Klimov. Exploration by random network distillation. *arXiv preprint arXiv:1810.12894*, 2018. doi: 10.48550/arXiv.1810.12894. URL https://arxiv.org/abs/1810.12894.

L. Chen, K. Lu, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, and I. Mordatch. Decision transformer: Reinforcement learning via sequence modeling. *arXiv preprint arXiv:2106.01345*, 2021.

C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.

L. Chrisman. Reinforcement learning with perceptual aliasing: The perceptual distinctions approach. In W. R. Swartout, editor, *Proceedings of the Tenth National Conference on Artificial Intelligence (AAAI-92)*, pages 183–188. AAAI Press, 1992. URL https://dblp.org/rec/conf/aaai/Chrisman92.

X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su. Mind2web: Towards a generalist agent for the web. *arXiv:2306.06070*, 2023.

M. Janner, Y. Du, J. Tenenbaum, and S. Levine. Planning with diffusion for flexible behavior synthesis. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 9902–9915. PMLR, 2022. URL https://proceedings.mlr.press/v162/janner22a.html.

T. Kagaya et al. Rap: Retrieval-augmented planning with contextual memory for multimodal agents. *arXiv:2402.03610*, 2024.

E. Z. Liu et al. Reinforcement learning on web interfaces using workflow-guided exploration. *arXiv:1802.08802*, 2018.

PaddleOCR Team. PP-OCRv4 technical report. https://paddlepaddle.github.io/PaddleOCR/v2.9/ppocr/blog/PP-OCRv4_introduction.html, 2023.

D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell. Curiosity-driven exploration by self-supervised prediction. In D. Precup and Y. W. Teh, editors, *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 2778–2787. PMLR, 06–11 Aug 2017. URL https://proceedings.mlr.press/v70/pathak17a.html.

RapidAI Team. RapidOCR: Ocr toolbox. https://pypi.org/project/rapidocr-onnxruntime/1.4.4/, 2025. Version 1.4.4.

R. Sekar, O. Rybkin, K. Daniilidis, P. Abbeel, D. Hafner, and D. Pathak. Planning to explore via self-supervised world models. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 8583–8592. PMLR, 2020. URL https://proceedings.mlr.press/v119/sekar20a.html.

N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023. doi: 10.48550/arXiv.2303.11366. URL https://arxiv.org/abs/2303.11366.

D. Silver and J. Veness. Monte-carlo planning in large pomdps. In *Advances in Neural Information Processing Systems 23*, pages 2164–2172. Curran Associates, Inc., 2010. URL https://papers.nips.cc/paper/4031-monte-carlo-planning-in-large-pomdps.

D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, et al. Mastering the game of go without human knowledge. *Nature*, 550(7676): 354–359, 2017.

D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.

T. Veiga and J. Renoux. From reactive to active sensing: A survey on information gathering in decision-theoretic planning. *ACM Computing Surveys*, 55(13s):280:1–280:22, 2023. doi: 10.1145/3583068. URL https://dl.acm.org/doi/10.1145/3583068.

T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao, T. J. Hua, Z. Cheng, D. Shin, F. Lei, Y. Liu, Y. Xu, S. Zhou, S. Savarese, C. Xiong, V. Zhong, and T. Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *arXiv preprint arXiv:2404.07972*, 2024. URL https://arxiv.org/abs/2404.07972.

S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022. doi: 10.48550/arXiv.2210.03629. URL https://arxiv.org/abs/2210.03629.

A. Zhou, K. Yan, M. Shlapentokh-Rothman, H. Wang, and Y.-X. Wang. Language agent tree search unifies reasoning acting and planning in language models. *arXiv preprint arXiv:2310.04406*, 2023a. doi: 10.48550/arXiv.2310.04406. URL https://arxiv.org/abs/2310.04406.

S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig. Webarena: A realistic web environment for building autonomous agents. *arXiv:2307.13854*, 2023b.

A Reproducibility details

A.1 Reported-run configuration

Table 9: Retrieval and one-step PUCT configuration used for the reported runs.

Component	Configuration
Metadata filter	Display mode and text-size bin
Sparse retrieval	BM25 over spatial control/text atoms
Dense retrieval	`text-embedding-3-small` with HNSW ANN
Fusion	RRF, fused top 200
Verification	0.5 control + 0.5 text Jaccard; $\tau = 0.93$
PUCT	$c_p = 1.0$
Reward	$\lambda_{\text{state}} = \lambda_{\text{edge}} = \lambda_{\text{amb}} = 1.0$
Ambiguity	$\kappa = 20$, $u_0 = 0.5$

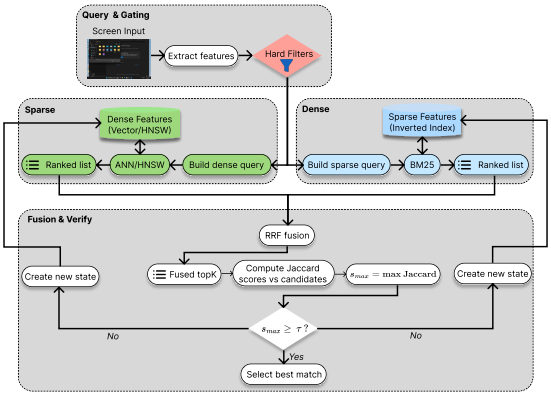


Figure 5: Hybrid screen retrieval. Metadata filters define eligibility; sparse and dense ranks generate candidates; bounded structural overlap makes the final merge decision.

Clicking Save can be represented as

`focus_click@1::r12_c7|button|save|notepad::none`,

whereas typing “hello” is

`focus_type@1::r15_c15|edit|text_editor|notepad::{"text":"hello"}`.

Changing “hello” to “world” changes the signature and therefore its edge and ambiguity statistics.

A.2 Reactive LLM baselines

Reactive runs use GPT-5.4 nano or mini at temperature 0, provider-default top-p, and a 140-token output maximum, with no heuristic fallback. The prompt requires one provided candidate and forbids invented IDs. `ctx=Yes` includes the last eight executed actions; `ctx=No` omits action history. Both variants receive the instruction, active-window metadata, counters, and up to 60 candidates. They return a JSON object containing an action, candidate ID, optional text, and a short reason; the executor validates it before use.

B Original Notepad diagnostic

The original policy diagnostic uses three verified Notepad replay starts and a fixed 50-action budget. It is retained as a transparent slice-level comparison of reactive proposal quality and graph-based search, but its small scale is not used as cross-application evidence.

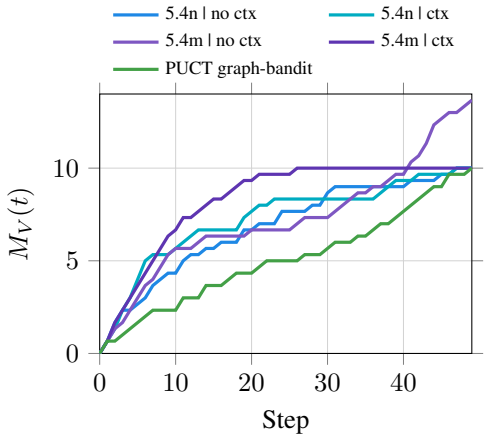


Figure 6: Frontier-state trajectories on the fixed three-start Notepad replay-start slice. Search continues to discover frontier states late in the budget, even after some reactive policies plateau.

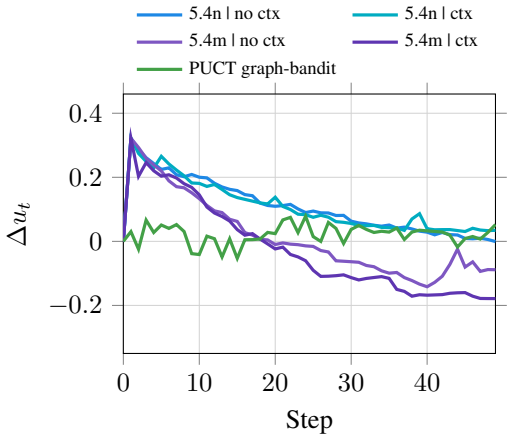


Figure 7: Ambiguity change on the same slice. Lower Δu_t means stronger disambiguation, not greater frontier growth.

B.1 Protocol and curve-derived metrics

Each episode begins from a verified replayable state reached from a clean snapshot by replaying a deterministic prefix. A state enters the slice only if the stored prefix reliably returns to the intended deduplicated state. We restrict the slice to eligible Notepad states with generation-mode provenance, at least three occurrences, and a completed replay anchor at step 5 or later, then deterministically select three starts.

We compare four reactive baselines that act directly from the current screen with the uncertainty-guided PUCT graph-bandit using the default uniform prior. Every episode uses steps 0–49, begins from the same replayed state, and receives the same public information. Runs are not initialized with states, edges, or search statistics discovered in earlier episodes. Keeping the PUCT prior uniform on this diagnostic separates graph-based selection from the deterministic heuristic and learned proposals evaluated elsewhere.

For a replay-start set $\mathcal{B}$, let $S_t^{(b)}$ be the raw observation at step t in episode b and let $s_t^{(b)} := \pi(S_t^{(b)})$ be its deduplicated state under the retrieval and deduplication map π. We define

$$M_V(t) := \frac{1}{|\mathcal{B}|} \sum_{b \in \mathcal{B}} \left| \{ s_\tau^{(b)} : 0 \le \tau \le t \} \right|, \qquad \Delta u_t := \frac{1}{|\mathcal{B}|} \sum_{b \in \mathcal{B}} \left(u(s_t^{(b)}) - u(s_0^{(b)}) \right). \tag{7}$$

We report $M_V(49)$ and Δu_{49} together with the discrete summaries $\sum_{t=0}^{49} M_V(t)$ and $\sum_{t=0}^{49} \Delta u_t$. Higher frontier values are better; lower ambiguity values indicate stronger movement toward lower-dispersion states.

Table 10: Exact frontier and ambiguity summaries derived from the fixed three-start Notepad replay-start slice.

Method	$M_V(49)\uparrow$	Frontier AUC↑	$\Delta u_{49}\downarrow$	Ambiguity AUC↓
Reactive GPT-5.4 nano, no ctx	10.00	340.33	−0.00	5.37
Reactive GPT-5.4 nano, ctx	10.00	362.00	0.03	5.41
Reactive GPT-5.4 mini, no ctx	**13.67**	362.33	−0.09	0.55
Reactive GPT-5.4 mini, ctx	10.00	**411.00**	−0.18	−1.23
PUCT graph-bandit	10.00	253.33	0.05	1.08

Interpretation. Table 10 and Figs. 6 and 7 show substantial variation across policies on this slice. The reactive GPT-5.4 mini variants attain the largest frontier totals; the context-enabled mini policy

has the best frontier AUC and the largest final ambiguity reduction. The nano variants are weaker on both frontier metrics, while the uniform-prior PUCT graph-bandit is more conservative here but continues discovering states late in the budget. Frontier growth and ambiguity reduction should therefore be read as separate diagnostics, and this three-start slice is evidence about behavioral differences rather than uniform method dominance.

B.2 Learned-proposal control

The same diagnostic also supports a proposal-prior control. Replacing the uniform PUCT proposal with the learned diffusion proposal raises both state and edge discovery while reducing revisit and loop/stall rates.

Table 11: Additional learned-proposal control on the three-start Notepad diagnostic.

Selector	Prior	States	State AUC	Edges	Edge AUC	Revisit	Loop/stall
PUCT	Uniform	10.00	253.33	46.67	1,199.00	0.796	0.440
PUCT	Learned diffusion	14.33	406.67	49.00	1,225.00	0.708	0.253

C Ambiguity audits

For 1,588 repeated state–action groups, we decompose pooled dispersion into within-recorded-context and between-recorded-context contributions.

Table 12: Context decomposition of repeated matched-action dispersion.

Application	Groups	Pooled	Within context	Between context
Notepad	715	0.158	0.005	0.152
File Explorer	611	0.530	0.001	0.529
LibreOffice Calc	262	0.120	0.000	0.120

The decomposition is descriptive. Recorded context may correlate with hidden workflow state, but it is not a complete latent-state annotation; unrecorded asynchronous effects and state-matching errors remain possible.

D Hyperparameter sensitivity

Table 13 gives the complete metric set for the component suite condensed in Table 5.

Table 13: Full three-application macro-average component metrics.

Configuration	New states	State AUC	New edges	Edge AUC	Net Δu	High-u occ.	Revisit	Loop/stall
Full: novelty + ambiguity	1.573	50.487	25.727	716.027	0.0083	0.6568	0.9678	0.8593
Novelty only	1.427	46.347	31.153	901.533	-0.0346	0.4907	0.9705	0.8828
Ambiguity only	1.047	34.300	26.640	761.987	0.0050	0.6818	0.9786	0.9000
Prior only	2.233	71.867	36.300	982.900	0.0218	0.7068	0.9544	0.8153
Full + uniform prior	4.733	132.527	41.880	1,094.720	0.0343	0.7983	0.9028	0.6937
Full + exact matching	2.100	67.847	36.227	997.240	-0.0155	0.6038	0.9570	0.8651

Each row in Table 14 varies one parameter while holding all other settings fixed in the matched three-application suite.

Table 14: One-factor-at-a-time sensitivity.

Parameter	Values	New-state AUC	Diagnostic
τ	0.90 / 0.93 / 0.96	54.573 / 50.487 / 33.847	False merge 0.009 / 0.005 / 0.004
κ	5 / 20 / 50	46.901 / 50.487 / 47.353	Revisit 0.9714 / 0.9678 / 0.9688
u_0	0.25 / 0.50 / 0.75	31.073 / 50.487 / 65.307	Revisit 0.9781 / 0.9678 / 0.9554

For $\tau = 0.90/0.93/0.96$, pair F1 is 0.632/0.622/0.594 and net ambiguity change is 0.0164/0.0083/-0.0371. Higher τ reduces false merges but creates more fragmented identities; the persisted UIA IDs used for pair F1 are a matcher diagnostic, not independent semantic ground truth.

E Alternative observation representation

We re-encode 1,200 stored screenshots—450 Notepad, 450 File Explorer, and 300 LibreOffice Calc—using offline UIA features or spatially localized OCR text. Existing SCREENSEARCH state IDs serve as reference labels. Within each application and representation, 40% of pairs select a cutoff that is frozen for the remaining 60%. RapidOCR 1.4.4 with PP-OCRv4 returns non-empty OCR for every screenshot.

Table 15: UIA and OCR representations on matched stored screenshots and actions.

Representation	Pair F1	Recall@10	False merge	Successor consistency
Offline UIA	0.740	0.381	0.010	0.892
OCR only	0.646	0.409	0.007	0.958

F Downstream training details

Task descriptions are synthesized with `gpt-4o-mini`; the model is used for label generation rather than as the PUCT selector. The reactive GPT-5.4 baselines described in Section A are evaluation comparators.

Table 16: Downstream training-data and evaluation properties.

Property	Value
Three-step trajectory windows	22,028
Unique task descriptions	8,118
Semantically unique descriptions	75.4%
Windows containing a state change	93.99%
Frozen evaluation prompts	1,000

Each action stores operation, target role/text, position, and optional argument. The model predicts the next typed action from the task and current screen. Counterfactual pairwise supervision pairs examples with the same task but different screens and encourages different actions when the GUI state requires them. Step Action Match is exact agreement with the recorded action. Screen Sensitivity is the accuracy decrease after shuffling screen context; State-Change Sensitivity applies the same test to windows containing a GUI state change. Offline three-step success and post-change recovery use recorded next screens after each prediction and are therefore teacher-forced proxies.

F.1 Software assets and licenses

RapidOCR and PaddleOCR are used under the Apache License 2.0. The Visual Studio Code source repository is MIT-licensed, and LibreOffice is distributed under MPL-2.0/LGPLv3+ terms. Commercial Microsoft applications and OpenAI API models are accessed under the authors' applicable organizational licenses and service terms. We do not redistribute third-party application code or model weights.

G Additional corpus diagnostics

G.1 Screenshot post-processing

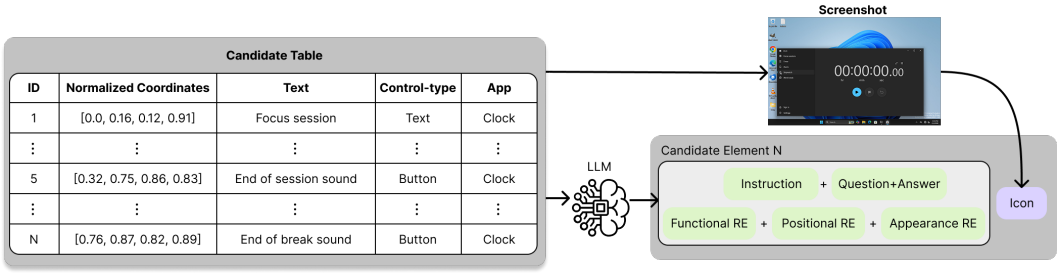


Figure 8: Overview of screenshot post-processing pipeline.

Each exploration step logs a raw screenshot alongside its structured UIA-derived candidate table, enabling fully offline annotation over the collected corpus. As shown in Figure 8, we then run a lightweight screenshot post-processing pipeline that takes the candidate table as input and emits a compact set of textual labels attached to the screenshot. These labels include a natural-language instruction, a question–answer pair, and short rationales that explain the screen in functional, positional, and appearance terms. We additionally map screenshot regions to icon instances in the candidate table to generate labels that are explicitly grounded to a particular icon.

Importantly, this pipeline does *not* require a vision-language model: the only model dependency is an LLM used to generate text labels from the structured UI metadata. Because post-processing is decoupled from exploration and operates over normalized, schema-consistent artifacts, new labelers and additional post-processing procedures can be implemented as standalone passes and replayed over existing data without re-running VM rollouts. The exact prompt templates are provided in Appendix G.2.

G.2 Screenshot processing prompts

Table 17: Prompt templates for icon/element labeling. Each prompt is conditioned on the UIA-derived candidate table. The model receives one or more candidate rows describing the target icon/element.

Component	Prompt Template
Caption	ID | Type | Text content or description | Normalized location [x1, y1, x2, y2] <CANDIDATE_ITEM_1> *(e.g., <ID> <TYPE> <TEXT_OR_DESC> [<x1>,<y1>,<x2>,<y2>])* The attached text data describes a specific icon/element present in a computer screen. Provide a concise 1-2 sentence caption describing this icon/element with respect to the entire computer screen.
Instruction	ID | Type | Text content or description | Normalized location [x1, y1, x2, y2] <CANDIDATE_ITEM_1> The attached text data describes a specific icon/element present in the computer screen. Provide a concise 1-2 sentence instruction describing how to use this icon/element with respect to the entire computer screen.
Question+Answer	ID | Type | Text content or description | Normalized location [x1, y1, x2, y2] <CANDIDATE_ITEM_1> The attached text data describes a specific icon/element present in the computer screen. Provide a concise natural language question a user might ask about using this icon/element. Also, provide the expected answer to the question.
All REs	ID | Type | Text content or description | Normalized location [x1, y1, x2, y2] <CANDIDATE_ITEM_1> I will provide data above about a specific icon/element to help you understand it. Your task is to classify the target area and generate several references regarding the target area on the original computer screen image. Specific task requirements are as follows: You will analyze and output a dictionary in JSON file format. The key-value pairs included are as follows: - functional_reference: A reference about the target area, involving the function of the target area. - positional_reference: A reference about the target area by describing the position of the target area, such as layout and nearby elements. - appearance_reference: A reference about the target area by describing the appearance of the target area. Ensure that anyone can uniquely identify this area in the screenshot through any one of the references. Ensure that the reference is complete and independent. Your output should follow this format strictly: {''Output'': {''functional_reference'': ''<...>'', ''positional_reference'': ''<...>'', ''appearance_reference'': ''<...>''}}

G.3 Action tokenization

G.4 Token normalization and rule-based semantic/value classes

Text normalization. All role/semantic strings are normalized by lowercasing, collapsing whitespace, and truncating to 120 characters.

Value-class mapping for `type`. Given a normalized typed string u, we map:

- `val_email` if u contains "@" or the substring "email";
- `val_password` if u contains "pass" or "pwd";
- `val_search` if u contains "search", "find", or "query";

Class c_t	Matched text/semantic items (substring triggers)
search	search box, find a setting, show all results, expand search box, search, find, query
window_control	minimize settings, maximize settings, restore settings, close settings, minimize, maximize, restore
navigation	open navigation, back, home
settings_recommendation	recommended settings, commonly used settings, optimize your settings
settings_network	network & internet, ethernet, bluetooth, dns, gateway, ipv4, ipv6, adapter, internet
settings_update	windows update, update history, up to date
settings_personalization	personalization, background, theme, color mode, dark, lock screen, pictures
settings_accessibility	accessibility, text size, text cursor, cursor thickness, make text size bigger
settings_accounts	accounts, sign-in options, windows hello, family, other users, local account, profile picture
settings_apps	installed apps, apps
settings_time_language	time & language
settings_privacy_security	privacy & security
settings_gaming	gaming
settings_printers	printers & scanners
settings_system	system
settings_device_info	device info, specifications, manufacturer, driver version, physical address
status_offline	no internet, not connected, no_internet
action_delete	delete
action_rename	rename
action_browse	browse
email	email, @
password	password, pwd
continue	continue, submit, next
signin	sign in, log in, login, sign-in
cancel	cancel
confirm	confirm, ok
save	save
open	open
close	close

Table 18: Rule-based mapping from target semantic text to c_t (first-match wins; fallback generic).

- otherwise val_generic_text.

Position binning from location. If location matches r<i>_c<j> with $i, j \in [0, 29]$, we bin:

$$\texttt{row_bin}(i) = \texttt{top}\ (i < 10),\ \texttt{mid}\ (10 \le i < 20),\ \texttt{bot}\ (i \ge 20),$$

$$\texttt{col_bin}(j) = \texttt{left}\ (j < 10),\ \texttt{center}\ (10 \le j < 20),\ \texttt{right}\ (j \ge 20),$$

and emit row_bin_col_bin; otherwise unknown.

Semantic class mapping for c_t. We derive c_t from the semantic string associated with the action target using a small set of special cases, regex detectors, and a fixed substring-based taxonomy (default generic).

- Special cases: settings $\rightarrow$ settings_root; glyph-only strings $\rightarrow$ icon.
- Network patterns: IPv4 / MAC / IPv6-like strings $\rightarrow$ network_value.
- Otherwise, we assign the first matching class in Table 18 via substring rules (fallback generic).

Table 19: Marginal frequencies for each action-slot token in the training corpus.

Metric	Value
`x.op` **values**	
`x.op=click`	63,881 (96.67%)
`x.op=key`	2,150 (3.25%)
`x.op=unknown`	50 (0.08%)
`x.op=scroll`	3 (0.00%)
`x.delta.r` **values**	
`x.delta.r=text`	45,329 (68.59%)
`x.delta.r=image`	12,592 (19.05%)
`x.delta.r=button`	5,868 (8.88%)
`x.delta.r=edit`	2,150 (3.25%)
`x.delta.r=pane`	73 (0.11%)
`x.delta.r=window`	50 (0.08%)
`x.delta.r=menuitem`	19 (0.03%)
`x.delta.r=list`	3 (0.00%)
`x.delta.c` **values**	
`x.delta.c=icon`	11,242 (17.01%)
`x.delta.c=unlabeled_image`	11,178 (16.91%)
`x.delta.c=settings_network`	6,150 (9.31%)
`x.delta.c=settings_accounts`	5,394 (8.16%)
`x.delta.c=generic`	4,791 (7.25%)
`x.delta.c=settings_update`	4,164 (6.30%)
`x.delta.c=search`	3,443 (5.21%)
`x.delta.c=settings_apps`	3,248 (4.91%)
`x.delta.c=settings_system`	2,655 (4.02%)
`x.delta.c=settings_root`	2,221 (3.36%)
`x.delta.c=settings_time_language`	2,198 (3.33%)
`x.delta.c=settings_personalization`	1,986 (3.01%)
`x.delta.c=navigation`	1,481 (2.24%)
`x.delta.c=settings_accessibility`	1,409 (2.13%)
`x.delta.c=unlabeled_button`	1,323 (2.00%)
`x.delta.c=settings_privacy_security`	863 (1.31%)
`x.delta.c=settings_gaming`	570 (0.86%)
`x.delta.c=settings_sorting`	308 (0.47%)
`x.delta.c=settings_device_info`	221 (0.33%)
`x.delta.c=cancel`	219 (0.33%)
`x.delta.p` **values**	
`x.delta.p=top_left`	17,177 (25.99%)
`x.delta.p=mid_left`	14,176 (21.45%)
`x.delta.p=top_center`	7,709 (11.67%)
`x.delta.p=bot_left`	6,986 (10.57%)
`x.delta.p=bot_right`	4,542 (6.87%)
`x.delta.p=top_right`	4,365 (6.61%)
`x.delta.p=mid_center`	3,918 (5.93%)
`x.delta.p=bot_center`	3,732 (5.65%)
`x.delta.p=mid_right`	3,332 (5.04%)
`x.delta.p=unknown`	147 (0.22%)
`x.eta` **values**	
`x.eta=(empty)`	63,931 (96.74%)
`x.eta=computer.keyboard.write("text")`	2,150 (3.25%)
`x.eta=down:med`	3 (0.00%)

G.5 Action-token distribution

G.6 Confusion analysis

Slot	Class	Errors	Share	Class err. rate
r	text	41	46.6%	19.0%
r	image	26	29.5%	50.0%
r	button	14	15.9%	56.0%
r	edit	7	8.0%	100.0%
c	icon	26	22.2%	41.9%
c	unlabeled_image	21	17.9%	44.7%
c	settings_apps	10	8.5%	47.6%
c	settings_system	10	8.5%	55.6%
c	generic	9	7.7%	52.9%
p	top_left	25	21.2%	29.1%
p	mid_left	23	19.5%	41.8%
p	top_center	21	17.8%	60.0%
p	bot_left	15	12.7%	38.5%
p	bot_right	10	8.5%	37.0%

Table 20: Dominant error-driving classes by slot (shares are within-slot).

Slot	Confusion	Count	Share of slot errors
r	text→image	31	35.2%
r	image→text	26	29.5%
r	button→text	14	15.9%
r	text→button	10	11.4%
r	edit→text	5	5.7%
c	icon→unlabeled_image	7	6.0%
c	unlabeled_image→icon	5	4.3%
c	unlabeled_image→settings_system	4	3.4%
c	settings_apps→icon	4	3.4%
c	settings_accounts→search	3	2.6%
p	mid_left→top_left	8	6.8%
p	top_left→mid_left	8	6.8%
p	top_left→top_center	7	5.9%
p	bot_left→top_left	6	5.1%
p	mid_left→bot_center	5	4.2%

Table 21: Top within-slot confusions under best-of-k decoding.